MCT344
Industrial Robotics

Faculty of Engineering - Ain Shams University
International Credit Hours Engineering Programs
ICHEP

AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

International Credit Hours Engineering Programs (I.CHEP)

MCT344

Industrial Robotics

Third Milestone

Supervised By:

Dr.Shady Ahmed,

Eng.Mina Yousry,

Eng. Dina Zakaria.

Team Members

Name	ID
Philopateer Elkes	21P0240
Yahia Mohamed	21P0041

Table of Contents

Abstract: 5
1.0 Introduction 6
2.0CAD Model Import & URDF Conversion 7
3.0 Configuring ROS2 Controllers 9
4.0 Creating the Launch File 11
5.0. MOVEIT SETUP ASSISTANT 14
5.1 Collison Matrix 14
5.2 Planning Groups 15
5.3 Robot Poses 15
5.4 ROS2 Controllers 21
5.5 Moveit Controllers 21
6.0 CONFIGURATION FILE 22
7.0 PYTHON SCRIPT 24
8.0 Launch File 26
9.0 Simulation 28
10.0 Drive Link 30
11.0 Repo Link 30

List of figures:

Figure 1:CAD Model 7
Figure 2:Joint Names 8
Figure 3:Joint controller.yaml 9
Figure 4:Joint controller.yaml 2 10
Figure 5:Launch file 1 11
Figure 6:Launch file 2 12
Figure 7:Launch file 3 13
Figure 8:Robot in Gazebo 13
Figure 9:1 Collison Matrix 14
Figure 10:Planning Groups 15
Figure 11:Figure 11:Zero Pose 15
Figure 12:Pos 1 16
Figure 13:Pos2 17
Figure 14:Pose3 18
Figure 15:Pose4 19
Figure 16:Gripper Fully Closed 20
Figure 17:Gripper Fully Opened 20
Figure 18:ROS2 Controllers 21
Figure 19:5 Moveit Controllers 21
Figure 20:CONFIGURATION FILE 23
Figure 21:PYTHON SCRIPT 25
Figure 22:Launch File 27

Abstract:

This report presents the implementation of a complete motion planning and execution pipeline for a 6-degree-of-freedom (6-DOF) robotic manipulator using MoveIt 2 within the ROS 2 framework. The objective is to simulate and control the robot's motion by integrating key components such as URDF-based robot modeling, trajectory planning, and visualization through RViz and Gazebo. A customized configuration package is generated using the MoveIt Setup Assistant and adapted to ensure seamless operation across the simulation and planning environments. To validate the system, the robot is tasked with reaching a series of goal positions derived from the team members' student ID digits, introducing project-specific variations. Final outputs include a working simulation, visual documentation, a detailed report, and a demonstration video highlighting the robot’s ability to perform accurate and reliable trajectory execution.

1.0 Introduction

The primary objective of this project is to showcase the implementation of motion planning and execution for a six-degree-of-freedom (6-DOF) robotic manipulator using the MoveIt 2 motion planning framework within the Robot Operating System 2 (ROS 2) ecosystem. This milestone extends core concepts in robotic simulation and control by constructing a complete and functional motion planning pipeline. The pipeline encompasses essential components such as robot modeling, trajectory generation, planning algorithm integration, and real-time visualization.

To initiate the setup, the MoveIt Setup Assistant is utilized to create a configuration package specifically adapted to the robot's Unified Robot Description Format (URDF) model. This package includes necessary launch files, planning configurations, and controllers. Subsequent customization and verification ensure seamless compatibility with both RViz—used for visualizing motion plans and joint configurations—and Gazebo, which provides realistic, physics-based simulation of the robot's dynamics and environment interactions.

For performance evaluation, the robotic manipulator is programmed to move to a set of predefined target poses. These target positions are uniquely derived from the last two digits of each team member's student ID, adding a layer of uniqueness and individualization to each team's implementation. This also tests the robot's ability to handle arbitrary goal states while maintaining accurate inverse kinematics and trajectory planning.

2.0CAD Model Import & URDF Conversion

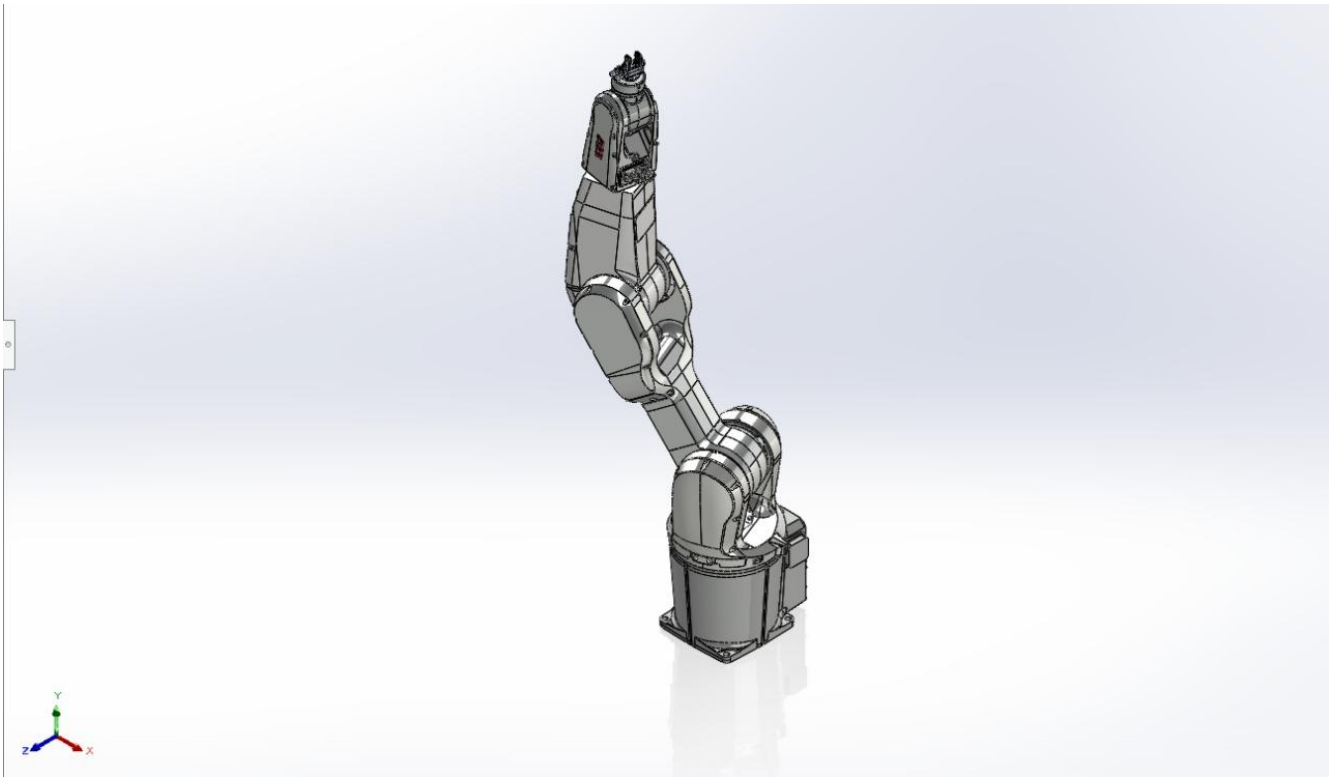


Figure 1:CAD Model

To integrate the robotic arm with ROS2, we exported the SolidWorks assembly as a URDF file using the SolidWorks URDF Exporter Plugin. During the export process, each link and joint was carefully named and assigned realistic mass and inertia values.

```xml
</hardware>
<joint name="joint1_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint2_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint3_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint4_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
    <param name="initial_value">0.0</param>
  </state_interface>
  <state_interface name="velocity">
    <param name="initial_value">0.0</param>
  </state_interface>
</joint>

<joint name="joint5_21P0041_21P0240">
  <command_interface name="position" />
  <state_interface name="position">
```

Figure 2:Joint Names

3.0 Configuring ROS2 Controllers

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100 # Hz
    arm_group_controller:
      type: joint_trajectory_controller/JointTrajectoryController
    hand_group_controller:
      type: joint_trajectory_controller/JointTrajectoryController
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

arm_group_controller:
  ros__parameters:
    joints:
      - joint1_21P0041_21P0240
      - joint2_21P0041_21P0240
      - joint3_21P0041_21P0240
      - joint4_21P0041_21P0240
    interface_name: position
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
    allow_nonzero_velocity_at_trajectory_end: true
    action_monitor_rate: 20.0
    allow_partial_joints_goal: false
    open_loop_control: true
    constraints:
      stopped_velocity_tolerance: 0.01
      goal_time: 0.0
      joint1_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint2_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint3_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03
      joint4_21P0041_21P0240:
        trajectory: 0.05
        goal: 0.03

hand_group_controller:
  ros__parameters:
    joints:
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
```

Figure 3:Joint controller.yaml

```yaml

hand_group_controller:
  ros__parameters:
    joints:
      - joint5_21P0041_21P0240
      - joint6_21P0041_21P0240
      - joint7_21P0041_21P0240
      - joint8_21P0041_21P0240
    interface_name: position
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
    allow_nonzero_velocity_at_trajectory_end: true
    gains:
      joint5_21P0041_21P0240: { p: 500.0, i: 10.0, d: 10.0 }
      joint6_21P0041_21P0240: { p: 500.0, i: 10.0, d: 10.0 }

```

Figure 4:Joint controller.yaml 2

4.0 Creating the Launch File

```python
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    IncludeLaunchDescription,
    SetEnvironmentVariable,
)
from launch.substitutions import PathJoinSubstitution
from launch_ros.actions import Node
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory
import xacro
import os


def generate_launch_description():
    pkg_ros_gz_sim = get_package_share_directory("ros_gz_sim")
    pkg_lab_gazebo = get_package_share_directory("abb_robot")

    # Process the Xacro file to generate URDF
    xacro_file = os.path.join(pkg_lab_gazebo, "urdf", "robot_pkg.urdf")
    doc = xacro.parse(open(xacro_file))
    robot_description = doc.toxml()

    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_ros_gz_sim, "launch", "gz_sim.launch.py")
        ),
        launch_arguments={"gz_args": "-r empty.sdf"}.items(),
    )

    # Robot State Publisher node
    robot_state_publisher_node = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        parameters=[{"robot_description": robot_description}],
        output="screen",
    )

    # Joint State Publisher node
    joint_state_publisher_node = Node(
        package="joint_state_publisher",
        executable="joint_state_publisher",
        name="joint_state_publisher",
        output="screen",
    )

    # Add the bridge node for mapping the topics between ROS and GZ sim
```

Figure 5:Launch file 1

```python
        package="ros_gz_bridge",
        executable="parameter_bridge",
        arguments=[
            # Clock (IGN -> ROS2)
            "/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock",
            # Joint states (IGN -> ROS2)
            "/world/empty/model/abb_robot/joint_state@sensor_msgs/msg/JointState[gz.msgs.JointState",
        ],
        remappings=[
            ("/world/empty/model/abb_robot/joint_state", "joint_states"),
        ],
        output="screen",
    )

    # Spawn Entity node
    spawn_entity = Node(
        package="ros_gz_sim",
        executable="create",
        arguments=["-name", "abb_robot", "-topic", "/robot_description"],
        parameters=[{"use_sim_time": True}],
        output="screen",
    )

    # add the robot controllers for joint trajectory, arm group and hand group controllers
    robot_controller = PathJoinSubstitution( [
        pkg_lab_gazebo,
        "config",
        "joint_controller.yaml", ]
    )

    # pass the controllers to the control node to send commands to the controllers
    control_node = Node(
        package="controller_manager",
        executable="ros2_control_node",
        parameters=[robot_controller],
        output="screen",
    )

    # Controller Manager Spawner node to launch the joint state broadcaster
    joint_state_broadcaster_spawner = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["joint_state_broadcaster", "--param-file", robot_controller],
    )
    # Controller Manager Spawner node to launch the arm group controller
    arm_controller_manager_spawner = Node(
```

Figure 6:Launch file 2

```python
        arguments=["joint_state_broadcaster", "--param-file", robot_controller],
    )
    # Controller Manager Spawner node to launch the arm group controller
    arm_controller_manager_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['arm_group_controller',"--param-file", robot_controller], output='screen'
    )
    # Controller Manager Spawner node to launch the hand group controller
    hand_controller_manager_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['hand_group_controller', "--param-file", robot_controller], output='screen'
    )

    # Get the meshes path in the install directory
    meshes_path = os.path.join(pkg_lab_gazebo, "meshes")
    return LaunchDescription(
        [
            SetEnvironmentVariable("GZ_SIM_RESOURCE_PATH", meshes_path),
            gazebo,
            spawn_entity,
            bridge,
            robot_state_publisher_node,
            joint_state_publisher_node,
            control_node,
            arm_controller_manager_spawner,
            hand_controller_manager_spawner,
            joint_state_broadcaster_spawner,
        ]
    )

```

Figure 7:Launch file 3

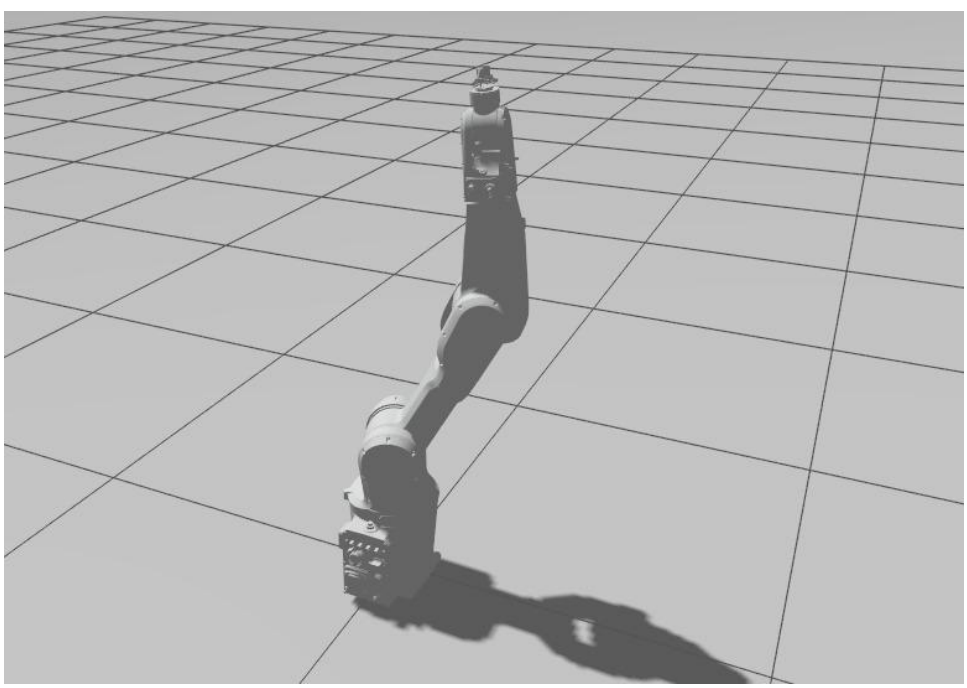


Figure 8:Robot in Gazebo

5.0. MOVEIT SETUP ASSISTANT

5.1 Collison Matrix

In this step, a collision matrix was configured to define the self-collision behavior between individual robot links.

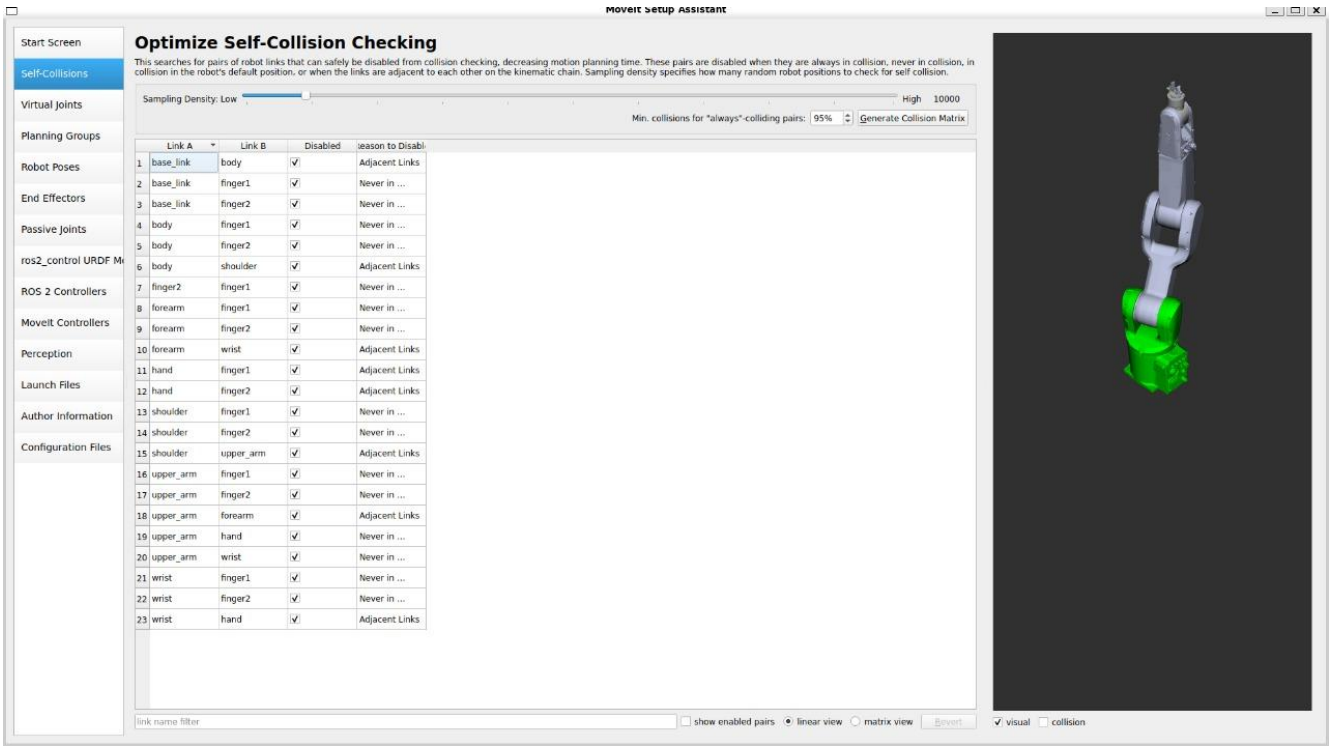


Figure 9:1 Collison Matrix

5.2 Planning Groups

Next, two planning groups were defined: one for the robotic arm, encompassing joints 1 through 6, and another for the hand (gripper), which includes joints 7 and 8.

Figure 10:Planning Groups

5.3 Robot Poses

In this section, five distinct poses were configured for the robot—three assigned to the arm group and two to the hand (gripper) group.

- Pose 0: This pose is defined as the initial position of the robot, so all joints are zero degrees (0 rad).

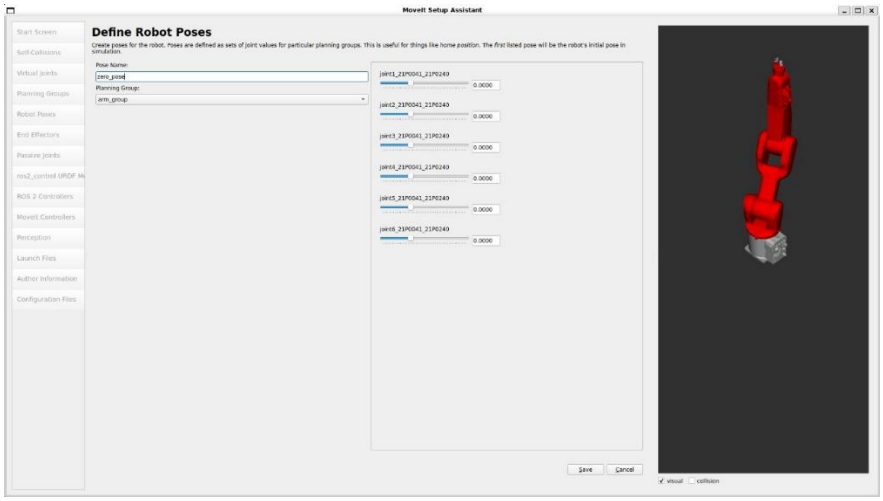


Figure 11:Figure 11:Zero Pose

<u>Pose 1</u>: In this pose the joints are defined with student id 21P0240

- **Joint 1**: 40° (0 0.6981 rad)
- **Joint 2**: 0 = 0° (0 rad)
- **Joint 3**: -20° (-0.3491 rad)
- **Joint 4**: 0°(0.0)
- **Joint 5**: 20°(0.1744 rad)
- **Joint 6**: 0° (0 rad)

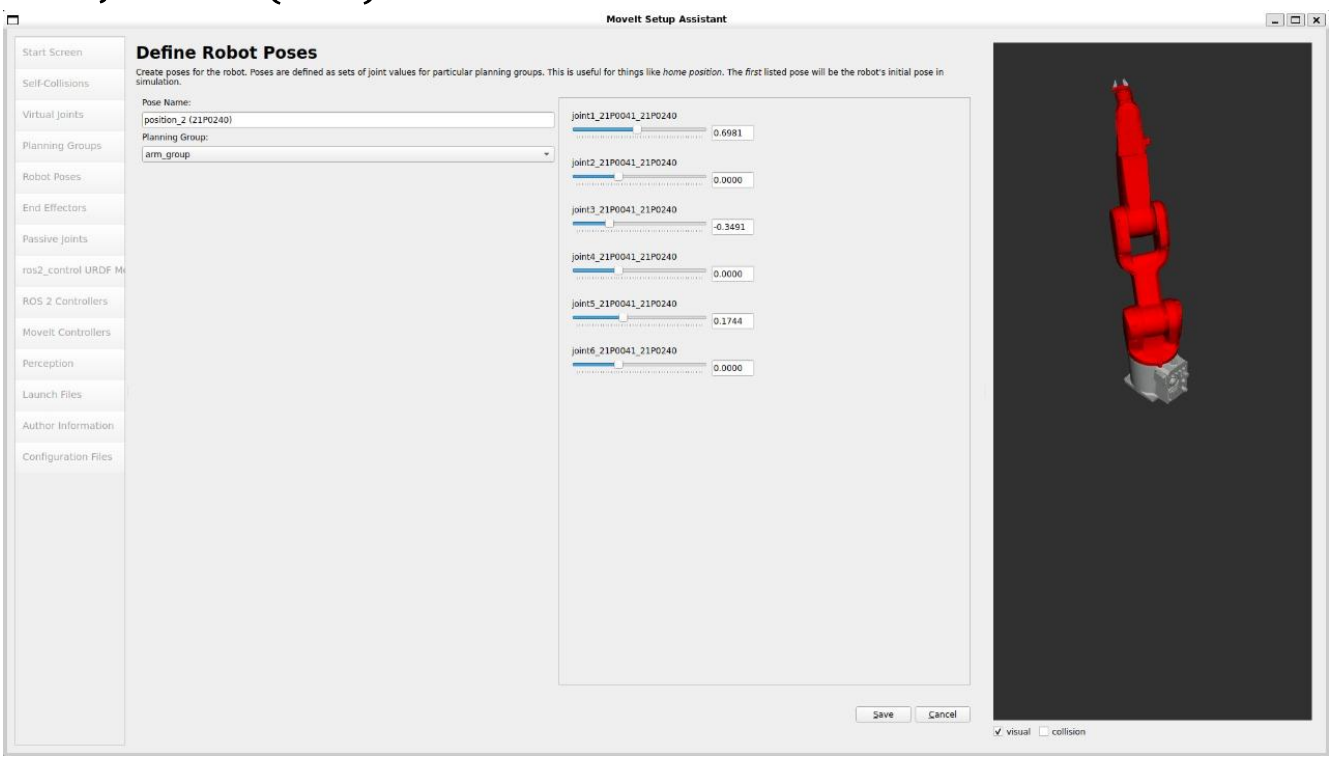


Figure 12:Pos 1

<u>Pose 2</u>: In this pose the joints are defined with student id 21P0041

- joint 1: 9°(0.1744)
- Joint 2: 40°(0.6981)
- Joint 3: 5°(0.0873)
- Joint 4: 20°(0.3491)
- Joint 5: 29°(0.5235)
- Joint 6: 0° (0 rad)

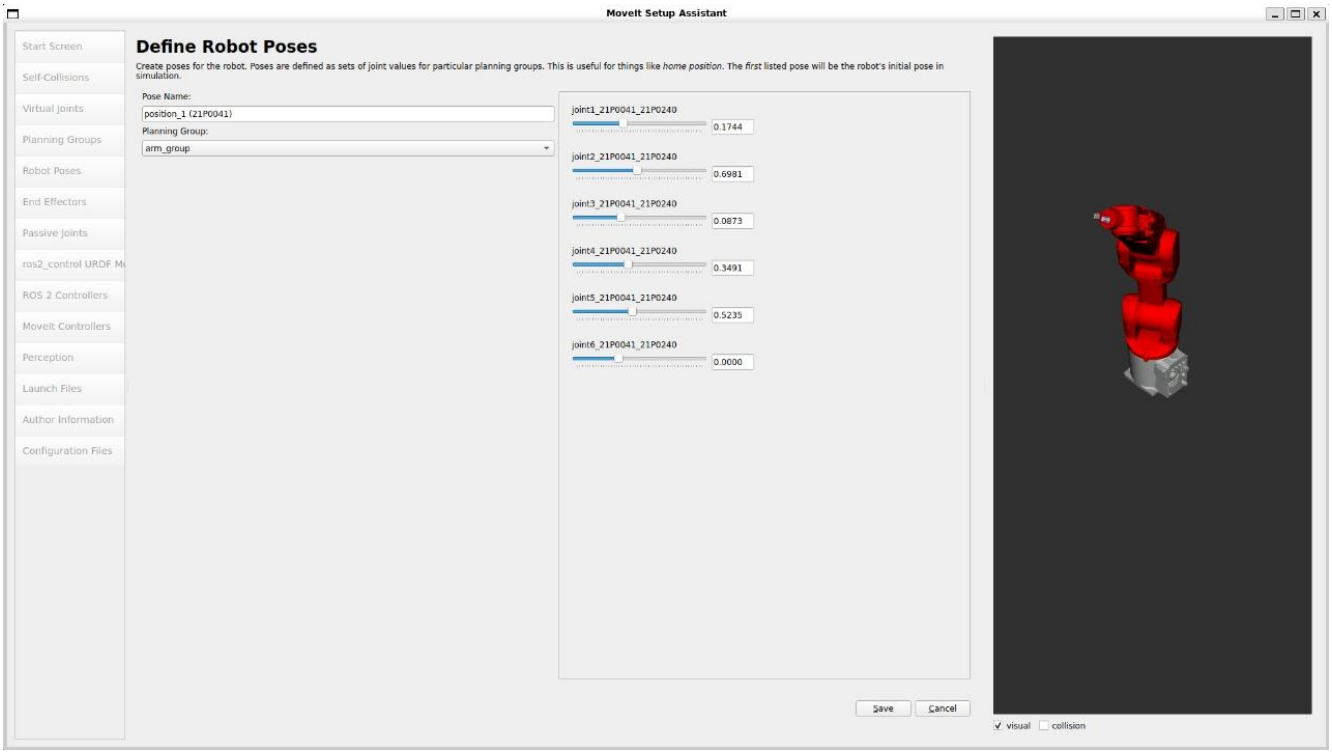


Figure 13:Pos2

Now we move to the new 2 poses for milestone 3:

Pos3_

- **joint1: 9° (**0.0872 rad)
- **joint2:** 16° (0.2792 rad)
- **joint3: 3°** (0.0523 rad)
- **joint4: 8°(**0.1396 rad)
- **joint5: 3°(**0.0523 rad)
- **joint6: 0°(**0.0000 rad)

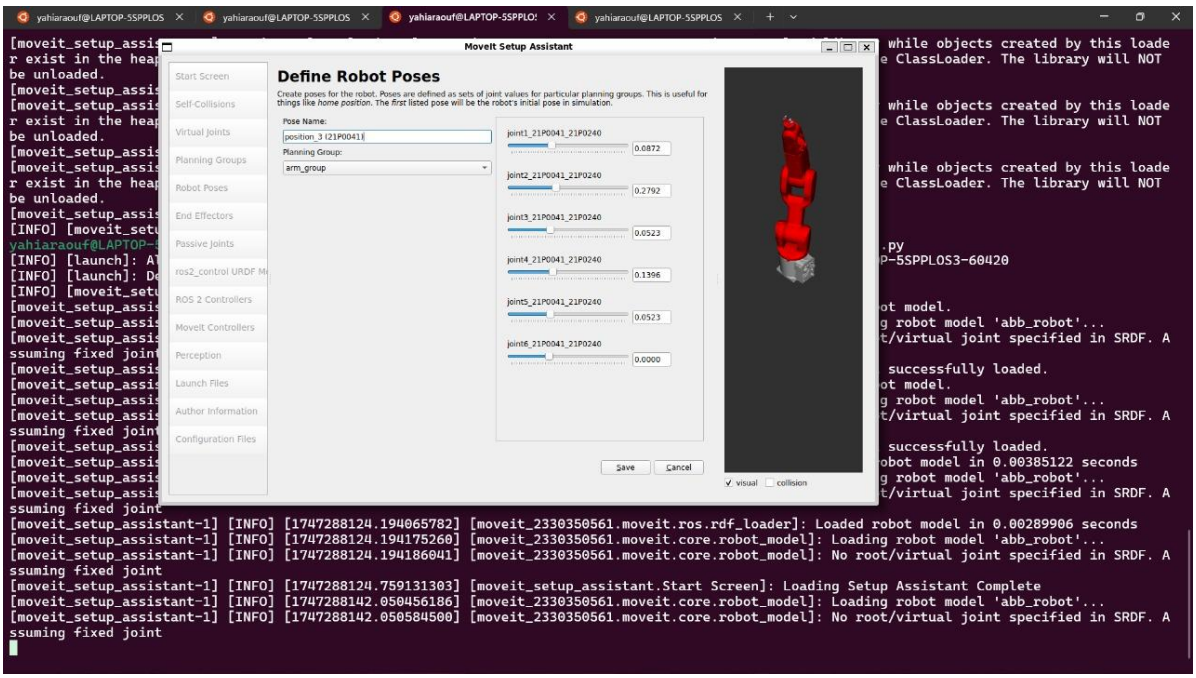


Figure 14:Pose3

Pos4_

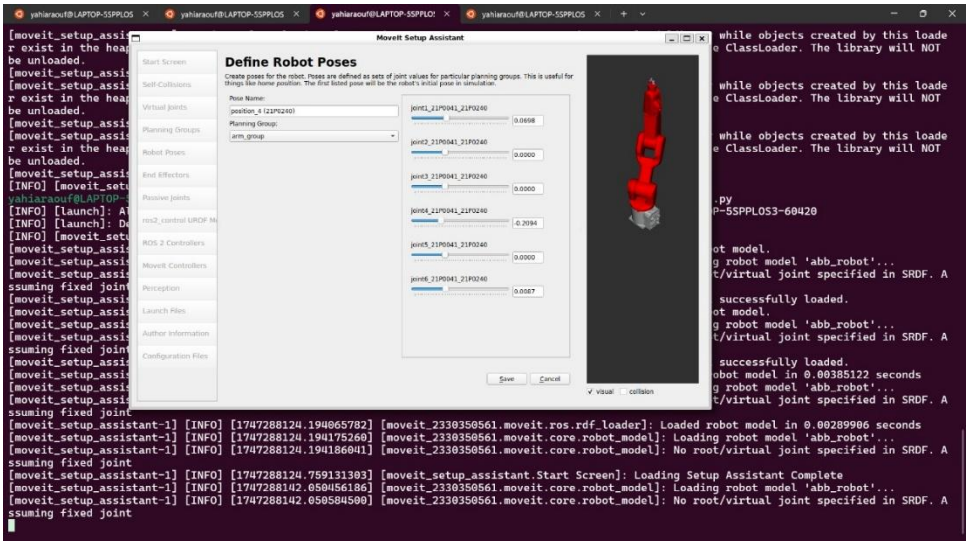


Figure 15:Pose4

Pose 4 (21P0240) — Degrees and Radians:

- **joint1:** 4° (0.0698 rad)
- **joint2:** 0° (0.0000 rad)
- **joint3:** 0° (0.0000 rad)
- **joint4:** -126.6° (–2.2094 rad)
- **joint5:** 0° (0.0000 rad)
- **joint6:** 0.5° (0.0087 rad)

- Close Hand pose: Gripper Fully closed

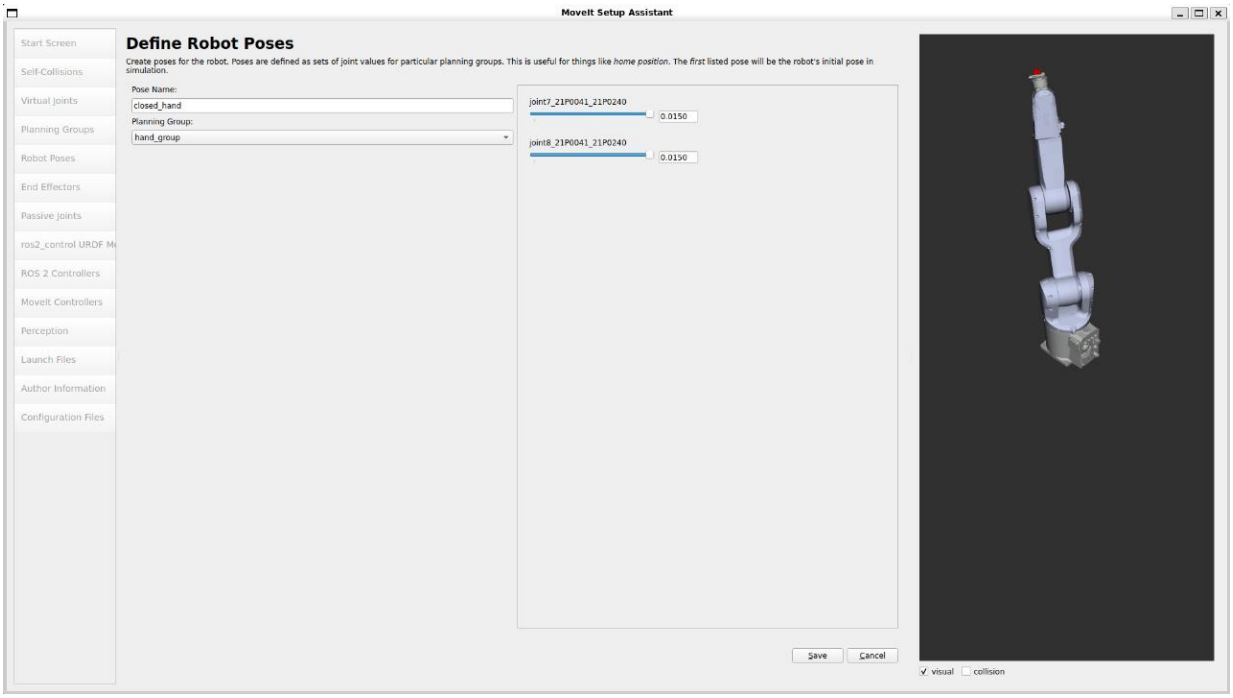


Figure 16:Gripper Fully Closed

- Open Hand Pose: Gripper Fully opened

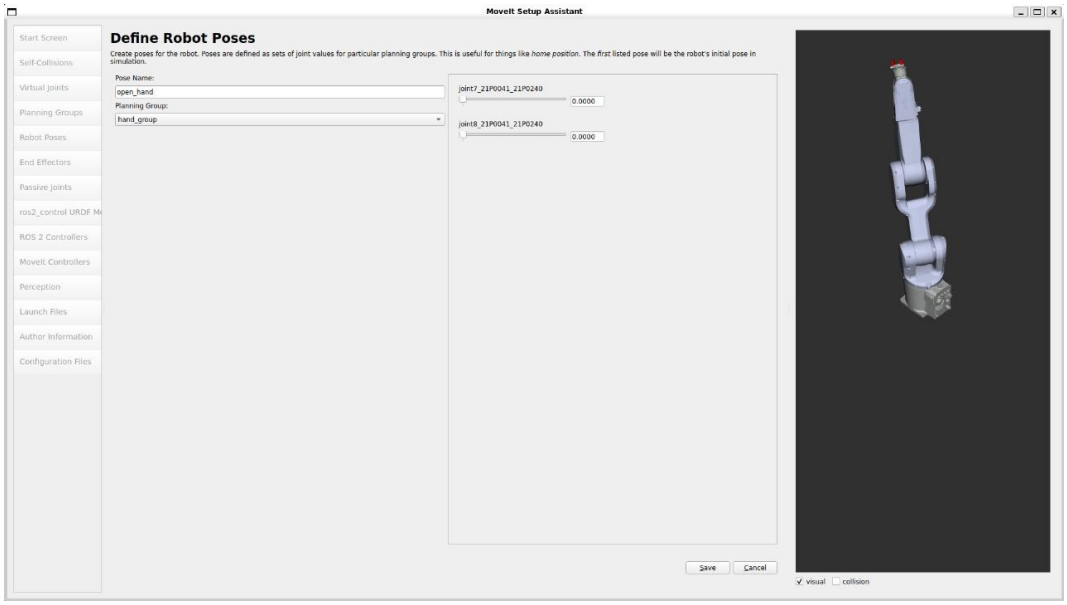


Figure 17:Gripper Fully Opened

5.4 ROS2 Controllers

In this step, we configured two ROS 2 controllers—one dedicated to the arm group and the other to the hand (gripper) group.

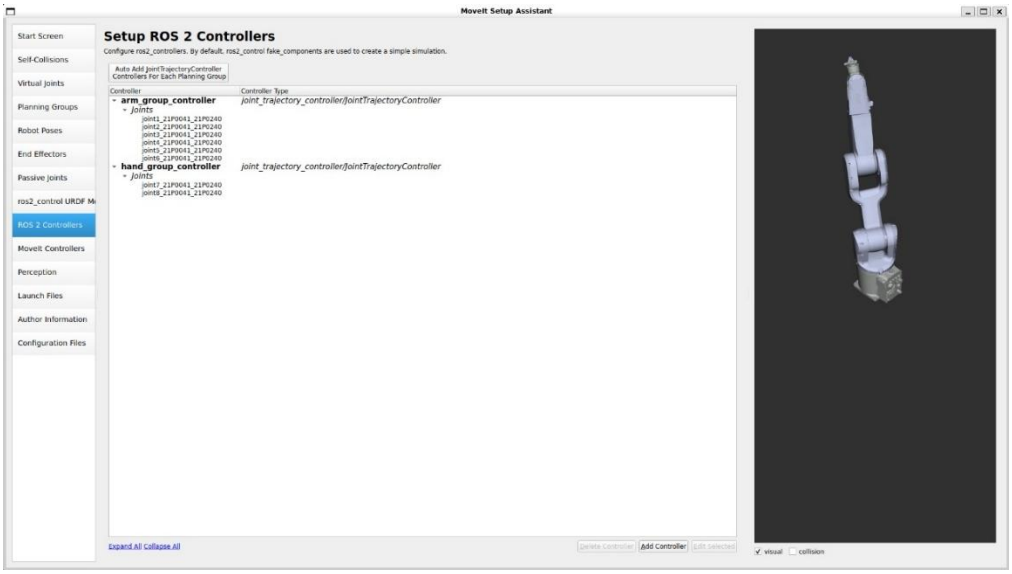


Figure 18:ROS2 Controllers

5.5 Moveit Controllers

Two MoveIt controllers were set up, each corresponding to one of the planning groups.

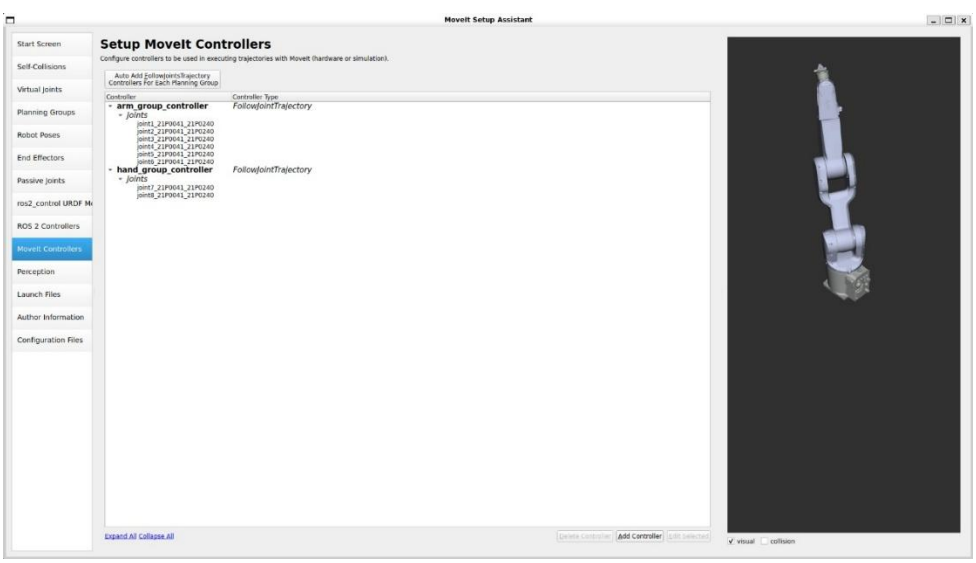


Figure 19:5 Moveit Controllers

6.0 CONFIGURATION FILE

This YAML configuration file is used to set up the motion planning system for a robot with MoveIt. It begins by specifying settings for the planning_scene_monitor, a crucial component that maintains an up-to-date internal model of the robot's environment. This is achieved by subscribing to key topics such as /joint_states to track the current positions of the robot's joints, ensuring that the planning scene stays synchronized with real-time data from both the robot and external sources. Next, the planning_pipelines section defines multiple available motion planners, giving the system flexibility in choosing the most appropriate one based on the specific motion task. These include:

- OMPL (Open Motion Planning Library): A widely used library for sampling-based planning strategies.
- Pilz Industrial Motion Planner: Designed for deterministic and safe industrial-style motion execution.
- CHOMP (Covariant Hamiltonian Optimization for Motion Planning): An optimization-based planner suitable for generating smooth, efficient paths.

The plan_request_params section sets general parameters for motion planning behavior. For instance, it limits the system to a single planning attempt per request and applies conservative velocity and acceleration scaling factors. These lower scaling values help ensure that planned motions are executed safely, reducing the risk of abrupt or unsafe movements. In this configuration, OMPL is specified as the default planner pipeline, meaning it will be used unless another planner is explicitly requested.

moveit_gazebo.launch.py 6 | robot_motion_planning_python.yaml

D: > college > Year 4 senior 1 > semseter 2 > Industrial Robotics > project > Industrial-Robotics-Project > Ros Packages > abb_moveit > config > robot_motion_planning_python.yaml

```yaml
planning_scene_monitor_options:
  name: "planning_scene_monitor"
  robot_description: "robot_description"
  joint_state_topic: "/joint_states"
  attached_collision_object_topic: "/moveit_cpp/planning_scene_monitor"
  publish_planning_scene_topic: "/moveit_cpp/publish_planning_scene"
  monitored_planning_scene_topic: "/moveit_cpp/monitored_planning_scene"
  wait_for_initial_state_timeout: 10.0
planning_pipelines:
  pipeline_names: ["ompl", "pilz_industrial_motion_planner", "chomp"]
plan_request_params:
  planning_attempts: 1
  planning_pipeline: ompl
  max_velocity_scaling_factor: 1.0
  max_acceleration_scaling_factor: 1.0
ompl_rrtc:
  plan_request_params:
    planning_attempts: 1
    planning_pipeline: ompl
    planner_id: "RRTConnectkConfigDefault"
    max_velocity_scaling_factor: 1.0
    max_acceleration_scaling_factor: 1.0
    planning_time: 1.0
pilz_lin:
  plan_request_params:
    planning_attempts: 1
    planning_pipeline: pilz_industrial_motion_planner
    planner_id: "PTP"
    max_velocity_scaling_factor: 1.0
    max_acceleration_scaling_factor: 1.0
    planning_time: 0.8

chomp_planner:
  plan_request_params:
    planning_attempts: 1
    planning_pipeline: chomp
    max_velocity_scaling_factor: 1.0
    max_acceleration_scaling_factor: 1.0
    planning_time: 1.5
```

Figure 20:CONFIGURATION FILE

7.0 PYTHON SCRIPT

This Python script is a ROS 2 motion control application that utilizes the MoveItPy interface to control a robotic arm and hand. It starts by initializing a ROS 2 node and creating planning components for two MoveIt groups: "arm_group" for the robotic arm and "hand_group" for the gripper.

At launch, the script moves the robotic arm to a predefined home position called pose_0, a safe and collision-free configuration defined in MoveIt. This ensures that the robot begins operation in a known and stable state. Once the arm is positioned, the script commands the gripper to close by setting its goal state to a named configuration close_hand, also defined in MoveIt under the hand group.

With the robot initialized and the hand closed, the script executes a reverse motion sequence through a list of predefined poses associated with students 21P0240 and 21P0041. The sequence includes the following poses: pose_2_21p0083, pose_1_21p0274, pose_4_21p0083, pose_3_21p0274, and finally back to pose_0. For each pose, a new goal state is assigned to the arm group, and the plan_and_execute function is used to generate and execute a motion plan.

The plan_and_execute function determines whether to use multi-pipeline or single-pipeline planning parameters, based on what's provided. It attempts to generate a valid plan using the specified planning method. If planning is successful, the robot executes the resulting trajectory;

```python
#!/usr/bin/env python3

import time
import rclpy
from rclpy.logging import get_logger

# moveit python library
from moveit.core.robot_state import RobotState
from moveit.planning import (
    MoveItPy,
    MultiPipelinePlanRequestParameters,
)


def plan_and_execute(
    robot,
    planning_component,
    logger,
    single_plan_parameters=None,
    multi_plan_parameters=None,
):
    # plan to goal
    logger.info("Planning trajectory")
    if multi_plan_parameters is not None:
        plan_result = planning_component.plan(
            multi_plan_parameters=multi_plan_parameters
        )
    elif single_plan_parameters is not None:
        plan_result = planning_component.plan(
            single_plan_parameters=single_plan_parameters
        )
    else:
        plan_result = planning_component.plan()
```

```python
def plan_and_execute(

    # execute the plan
    if plan_result:
        logger.info("Executing plan")
        robot_trajectory = plan_result.trajectory
        robot.execute(robot_trajectory, controllers=[])
        time.sleep(3.0)
    else:
        logger.error("Planning failed")


def main():
    rclpy.init()
    logger = get_logger("moveit_py.reverse_motion")

    moveit_node = MoveItPy(node_name="moveit_py")
    arm = moveit_node.get_planning_component("arm_group")
    hand = moveit_node.get_planning_component("hand_group")
    logger.info("MoveItPy instance created")

    # Move to zero_pose to initialize safely
    logger.info("Moving to zero_pose to initialize state")
    arm.set_goal_state(configuration_name="zero_pose")
    arm.set_start_state_to_current_state()
    plan_and_execute(moveit_node, arm, logger)

    # close hand before motion sequence
    logger.info("Closing hand before motion sequence")
    hand.set_goal_state(configuration_name="closed_hand")
    hand.set_start_state_to_current_state()
    plan_and_execute(moveit_node, hand, logger)

    # Step through poses in reverse order
    for pose_name in ["position_1 (21P0041)", "position_3 (21P0041)", "position_4 (21P0240)", "position_2 (21P0240)","zero_pose"]:
```

```python
def main():
    # Move to zero_pose to initialize safely
    logger.info("Moving to zero_pose to initialize state")
    arm.set_goal_state(configuration_name="zero_pose")
    arm.set_start_state_to_current_state()
    plan_and_execute(moveit_node, arm, logger)

    # close hand before motion sequence
    logger.info("Closing hand before motion sequence")
    hand.set_goal_state(configuration_name="closed_hand")
    hand.set_start_state_to_current_state()
    plan_and_execute(moveit_node, hand, logger)

    # Step through poses in reverse order
    for pose_name in ["position_1 (21P0041)", "position_3 (21P0041)", "position_4 (21P0240)", "position_2 (21P0240)","zero_pose"]:
        logger.info(f"Planning: current → {pose_name}")
        arm.set_goal_state(configuration_name=pose_name)
        arm.set_start_state_to_current_state()
        plan_and_execute(moveit_node, arm, logger)

    moveit_node.shutdown()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

Figure 21:PYTHON SCRIPT

8.0 Launch File

This ROS 2 launch file sets up the complete environment for robotic motion planning and control using the MoveIt 2 framework. It orchestrates the initialization of a Python-based motion planning script alongside essential components responsible for robot description, state publishing, and controller management.

The process begins by constructing the robot's MoveIt configuration through the MoveItConfigsBuilder. This utility loads the robot's description from a URDF file (robot.urdf.xacro) and imports motion planning settings from a YAML configuration file (robot_motion_planning_python.yaml). Additionally, it declares a launch argument example_file, which enables users to specify the name of the Python script that implements motion planning logic. If no file is provided, it defaults to robot_motion_python.py.

The launch file then initializes the following three core nodes:

- **robot_state_publisher:** This node publishes the robot's joint states and corresponding coordinate transforms (TFs) using the provided URDF model. It ensures that the current pose of each link in the robot is available to the ROS 2 ecosystem for visualization and control.
- **moveit_py_node:** This node executes the Python-based motion planning script through the MoveItPy interface. It leverages the robot's MoveIt configuration and planning parameters to perform tasks such as kinematic planning, trajectory generation, and motion execution.
- **ros2_control_node:** Responsible for launching the hardware interface and controller management system defined by the ROS 2 Control framework. It loads controller settings from the ros2_controllers.yaml **file.**

After initializing the main components, the launch file dynamically spawns the following controllers using the controller_manager spawner command:

- arm_group_controller – manages trajectory control for the robot's arm joints.
- hand_group_controller – handles motion control for the robot's end-effector or hand.
- joint_state_broadcaster – broadcasts the real-time state of each joint, enabling other components like RViz to visualize the robot's posture accurately.

robot_motion_planning.launch.py

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument, ExecuteProcess
from launch.substitutions import LaunchConfiguration
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
    moveit_config = (
        MoveItConfigsBuilder(
        robot_name="abb_robot", package_name="abb_moveit"
        )
        .robot_description(file_path="config/abb_robot.urdf.xacro")
        .moveit_cpp(
        file_path=get_package_share_directory("abb_moveit")
        + "/config/robot_motion_planning_python.yaml"
        )
        .to_moveit_configs()
    )

    example_file = DeclareLaunchArgument(
        "example_file",
        default_value="main.py",
        description="Python API tutorial file name",
    )

    moveit_py_node = Node(
        name="moveit_py",
        package="abb_moveit",
        executable=LaunchConfiguration("example_file"),
        output="both",
        parameters=[moveit_config.to_dict(),
        {"use_sim_time": True}],
```

robot_motion_planning.launch.py

```python
def generate_launch_description():
    moveit_py_node = Node(
        {"use_sim_time": True}],
    )

    robot_state_publisher = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        name="robot_state_publisher",
        output="log",
        parameters=[moveit_config.robot_description,
        {"use_sim_time": True}],
    )

    ros2_controllers_path = os.path.join(
        get_package_share_directory("abb_moveit"),
        "config",
        "ros2_controllers.yaml",
    )
    ros2_control_node = Node(
        package="controller_manager",
        executable="ros2_control_node",
        parameters=[ros2_controllers_path],
        remappings=[
            ("/controller_manager/robot_description", "/robot_description"),
        ],
        output="log",
    )
    load_controllers = []
    for controller in [
        "arm_group_controller",
        "hand_group_controller",
        "joint_state_broadcaster",
    ]:
        load_controllers += [
```

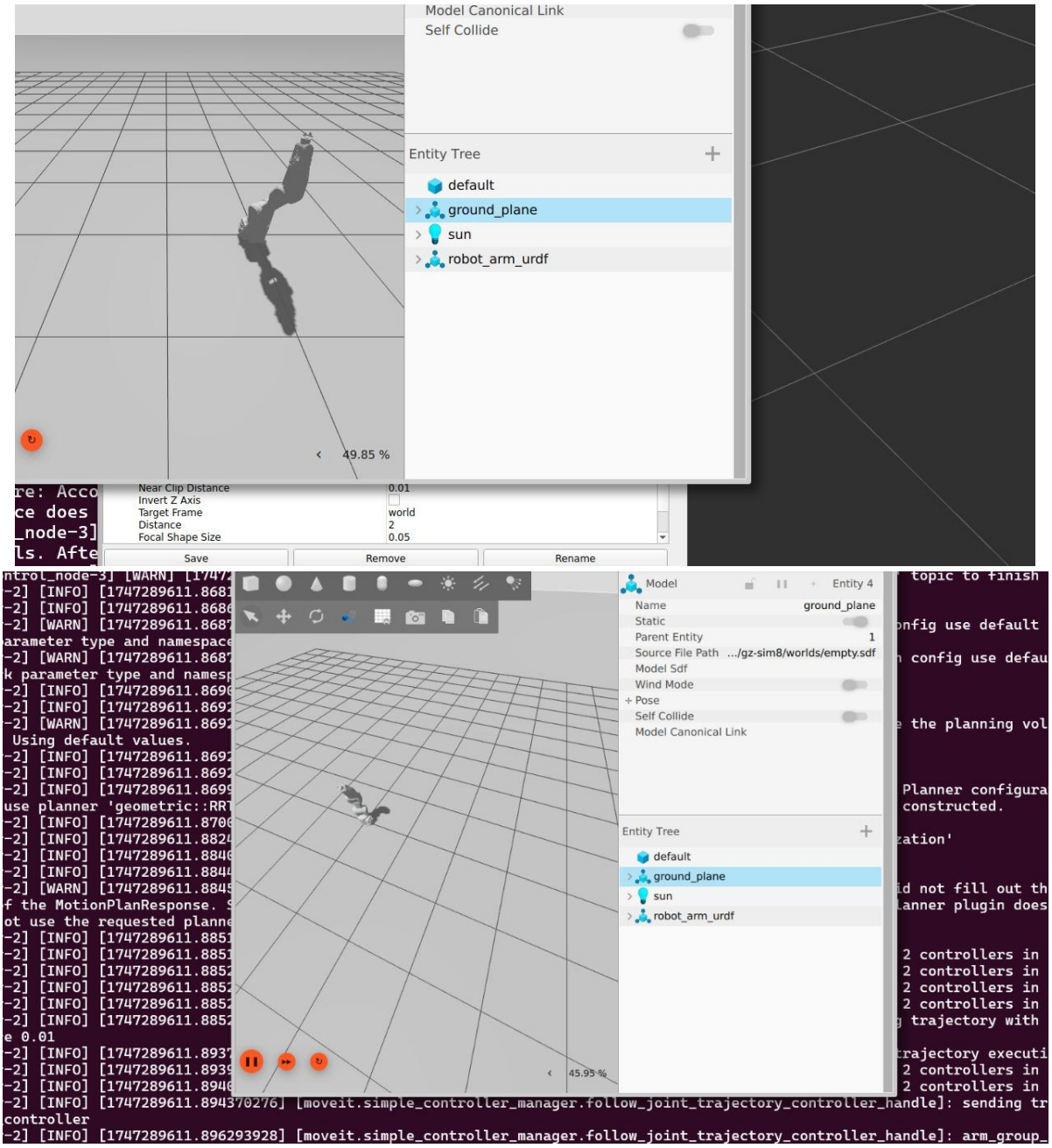
robot_motion_planning.launch.py

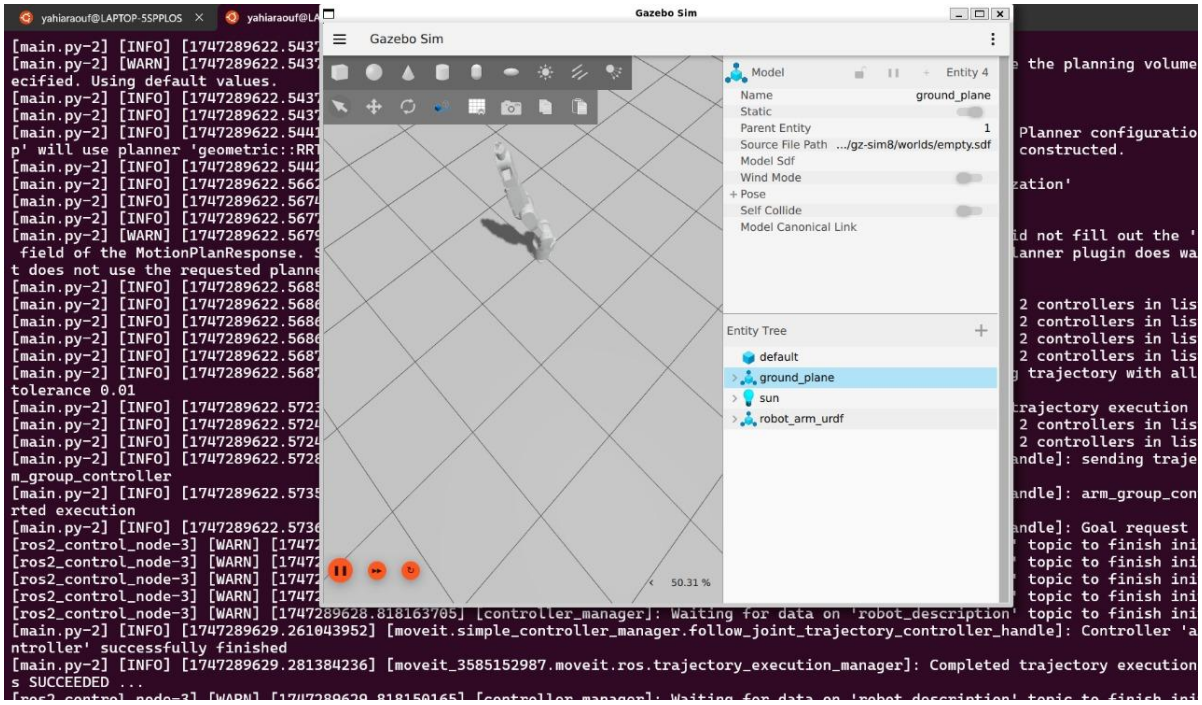
```python
def generate_launch_description():
    ros2_control_node = Node(
        ],
        output="log",
    )
    load_controllers = []
    for controller in [
        "arm_group_controller",
        "hand_group_controller",
        "joint_state_broadcaster",
    ]:
        load_controllers += [
            ExecuteProcess(
                cmd=["ros2 run controller_manager spawner {}".format(controller)],
                shell=True,
                output="log",
            )
        ]

    return LaunchDescription(
        [
            example_file,
            robot_state_publisher,
            moveit_py_node,
            ros2_control_node,
        ]
        + load_controllers
    )
```

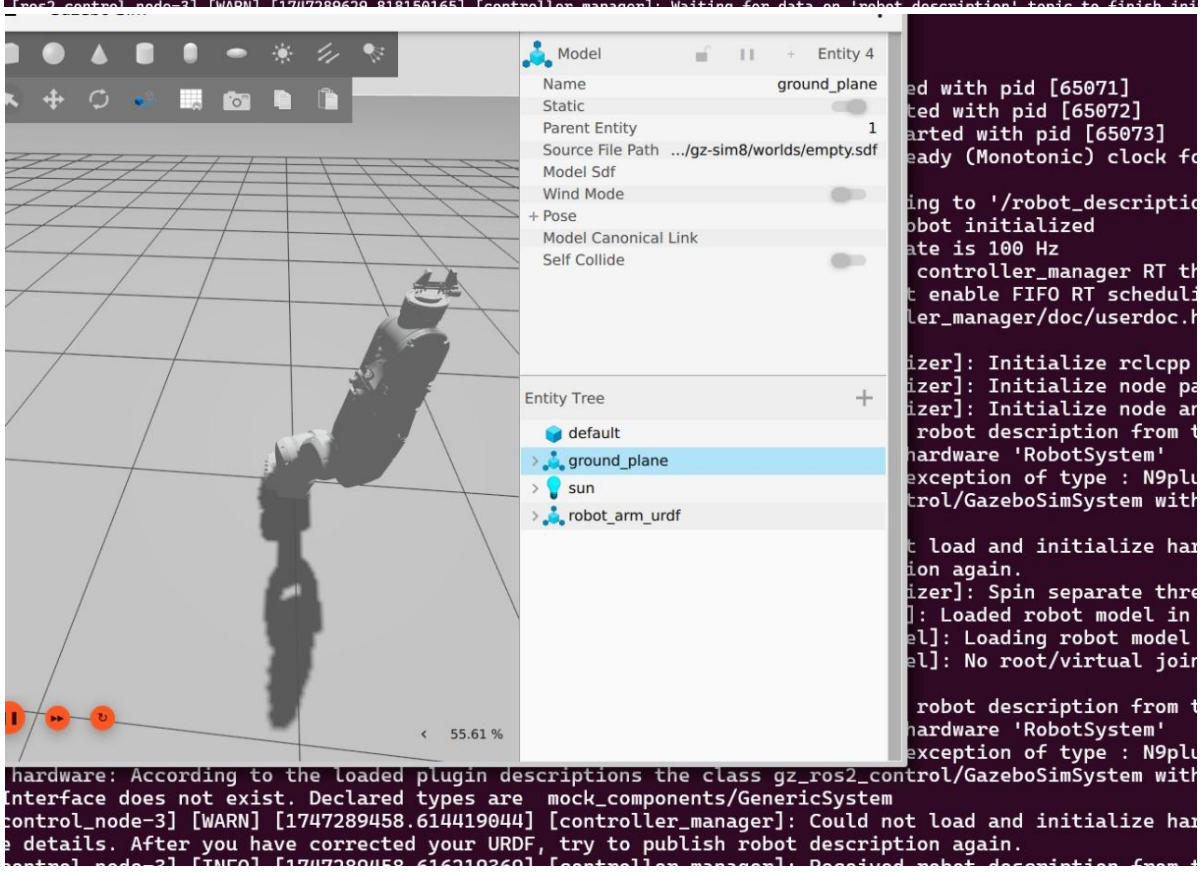
Figure 22:Launch File

9.0 Simulation

Here is a simulation for All poses.




yahiaraouf@LAPTOP-5SPPLOS
Gazebo Sim
Model
Entity 4
Name
ground_plane
Static
Parent Entity
1
Source File Path .../gz-sim8/worlds/empty.sdf
Model Sdf
Wind Mode
Pose
Self Collide
Model Canonical Link
Entity Tree
default
ground_plane
sun
robot_arm_urdf
50.31 %
[main.py-2] [INFO] [1747289622.5437
[main.py-2] [WARN] [1747289622.5437
ecified. Using default values.
p' will use planner 'geometric::RRT
field of the MotionPlanResponse.
t does not use the requested planne
tolerance 0.01
m_group_controller
rted execution
[ros2_control_node-3] [WARN] [1747289628.818163705] [controller_manager]: Waiting for data on 'robot_description' topic to finish ini
[main.py-2] [INFO] [1747289629.261043952] [moveit.simple_controller_manager.follow_joint_trajectory_controller_handle]: Controller 'a
ntroller' successfully finished
[main.py-2] [INFO] [1747289629.281384236] [moveit_3585152987.moveit.ros.trajectory_execution_manager]: Completed trajectory execution
s SUCCEEDED ...
the planning volume
Planner configuratio
constructed.
2 controllers in lis
trajectory with all
trajectory execution
handle]: sending traje
handle]: arm_group_con
handle]: Goal request
topic to finish ini


Model
Entity 4
Name
ground_plane
Static
Parent Entity
1
Source File Path .../gz-sim8/worlds/empty.sdf
Model Sdf
Wind Mode
Pose
Model Canonical Link
Self Collide
Entity Tree
default
ground_plane
sun
robot_arm_urdf
55.61 %
[ros2_control_node-3] [WARN] [1747289458.614419044] [controller_manager]: Could not load and initialize har
e details. After you have corrected your URDF, try to publish robot description again.
ed with pid [65071]
ted with pid [65072]
arted with pid [65073]
eady (Monotonic) clock fo
ing to '/robot_descriptio
obot initialized
ate is 100 Hz
controller_manager RT th
t enable FIFO RT scheduli
ler_manager/doc/userdoc.h
izer]: Initialize rclcpp
izer]: Initialize node pa
izer]: Initialize node ar
robot description from t
hardware 'RobotSystem'
exception of type : N9plu
trol/GazeboSimSystem with
t load and initialize har
ion again.
izer]: Spin separate thre
]: Loaded robot model in
el]: Loading robot model
el]: No root/virtual join
robot description from t
hardware 'RobotSystem'
exception of type : N9plu
hardware: According to the loaded plugin descriptions the class gz_ros2_control/GazeboSimSystem with
Interface does not exist. Declared types are mock_components/GenericSystem

10.0 Drive Link

https://drive.google.com/drive/folders/1MPEBC09B-SQcu0DGnX5WpHNOn8d4YA_R?usp=sharing

11.0 Repo Link

https://github.com/YahiaRaouf/Industrial-Robotics-Project.git